

Trabajar con el Modelo

De entender cómo piensa el modelo a saber dirigirlo:
contexto, prompts, estructura y criterio para evaluar.

1. La ventana de contexto
2. Prompt engineering: la base
3. Estrategias de estructuración
4. Una tarea vs. varias: cuándo dividir
5. Evaluar si una respuesta es buena

Navega con   o los botones de abajo

1

TEMA 1

La ventana de contexto

La mesa de trabajo del modelo: finita, y todo lo que pones encima cuenta.

La mesa de trabajo del modelo

ANALOGÍA · LA MESA DE TRABAJO

Un carpintero con una mesa de tamaño fijo: planos, herramientas y la pieza en la que trabaja — pero solo hasta donde quepa. Si amontona demasiado, lo del fondo queda tapado. La ventana es esa mesa: todo lo que el modelo necesita "ver" para responder tiene que caber a la vez.

La **ventana de contexto** es la cantidad máxima de tokens que el modelo procesa en una sola interacción.

EL DATO QUE SORPRENDE A UN TÉCNICO

El modelo **no tiene memoria entre llamadas**. En cada mensaje se le reenvía toda la conversación desde el inicio. Lo que parece "memoria" es el historial viajando una y otra vez dentro de la ventana.

Cuando la conversación ya no cabe, lo más antiguo se descarta o se resume. Lo que no está dentro, para el modelo no existe.

Todo cuenta, no solo "lo que escribo"

VENTANA DE CONTEXTO · p. ej. 200.000 tokens

[System prompt] instrucciones de la plataforma + rol + reglas

[Herramientas] definiciones de tools (búsqueda, código...)

[Documentos] archivos pegados, PDFs, código adjunto

[Historial] todos los turnos previos de la conversación

[Tu mensaje] la pregunta o instrucción actual

[Respuesta] lo que el modelo genera ahora – también ocupa

Seis fuentes compiten por el mismo espacio finito.

EL CAMBIO MENTAL

La ventana se mide en **tokens**, no en **mensajes**. Un mensaje corto con un PDF adjunto puede ocupar más que cincuenta mensajes de texto. Razona en tokens, no en turnos.

Costo, límite, latencia... y calidad

DIMENSIÓN	CÓMO LA AFECTA LA VENTANA
Costo	Se cobra por token de entrada y salida. Reenviar un historial gigante en cada turno multiplica el costo, aunque preguntes una línea.
Límite	Si lo que quieres meter no cabe, no entra. Hay que elegir qué incluir.
Latencia	Más tokens de entrada → respuesta más lenta. Una ventana medio llena va más despacio que una limpia.
Calidad	Contraintuitivo pero medido: más contexto puede empeorar la respuesta → siguiente slide.

CACHING

Las plataformas pueden **cachear** partes estables del contexto (un documento, un system prompt largo) para no pagarlas enteras en cada turno. Por eso reutilizar el mismo documento en una conversación sale más barato que pegarlo de nuevo.

Más contexto no es mejor contexto

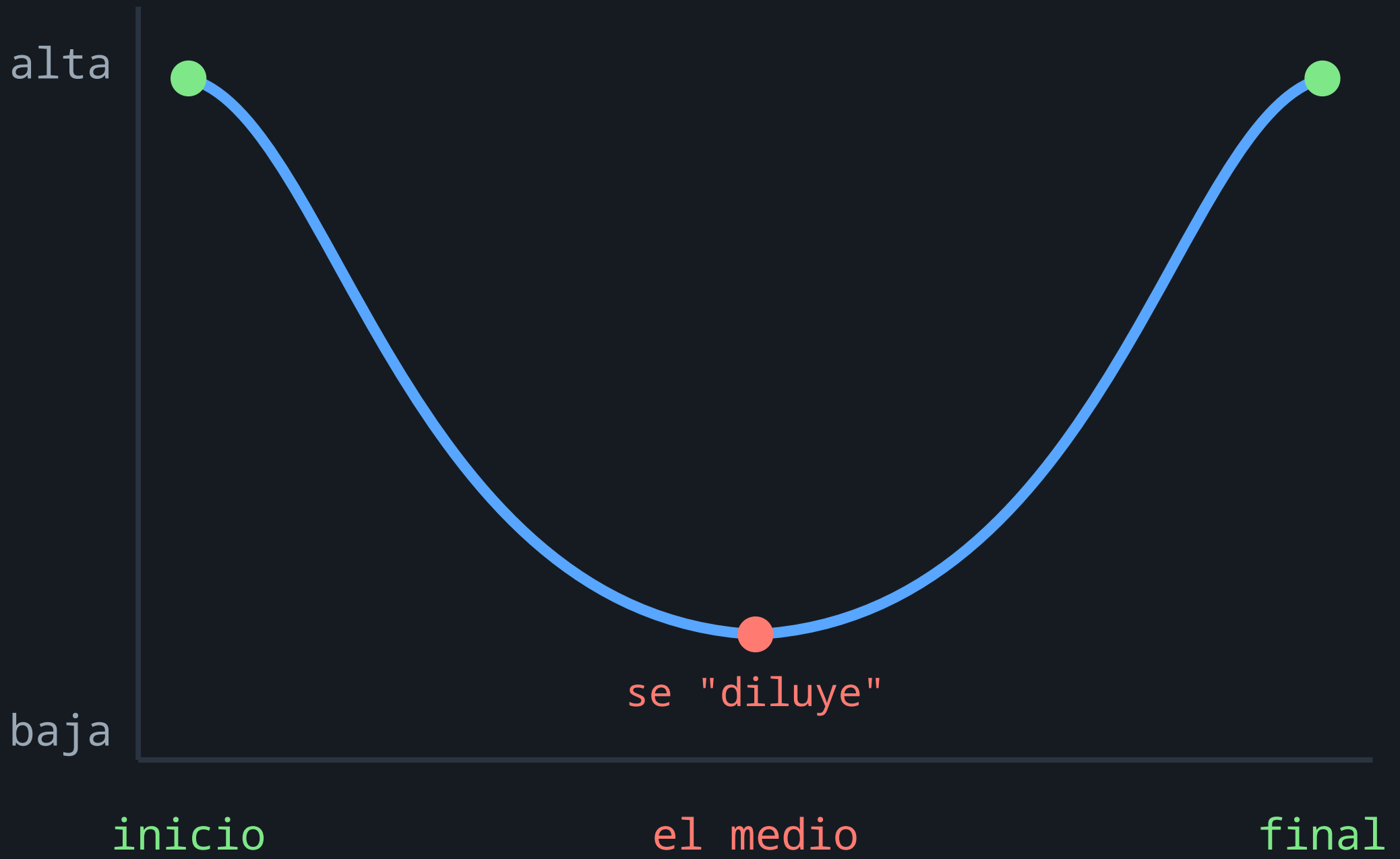
El rendimiento se **degrada** a medida que crece la entrada, aunque el dato relevante siga ahí. Se le llama *context rot*.

ANALOGÍA · EL CORREO ENTERRADO

Un dato en los últimos 3 correos lo encuentras al instante. El mismo dato en la posición 4.000 de 8.000 correos: técnicamente "lo tienes", pero casi nunca lo recuperas. El modelo sufre exactamente eso.

Lost in the middle: presta más atención al **inicio** y al **final** que al centro. Mover el dato del borde al medio puede bajar la precisión **más de un 30%**.

Degradación por longitud: Chroma (2025) probó 18 modelos de frontera y **todos** rindieron peor con entradas más largas. No es defecto de un modelo: es arquitectónico.



Precisión según la posición del dato.

IMPLICACIÓN PRÁCTICA

Pon lo importante **al principio o al final**. Con documentos largos: **documento arriba, pregunta al final** (mejora hasta 30% en pruebas de Anthropic). Y recuerda: **menos es más** — un contexto curado supera a uno enorme y ruidoso.

Estrategias para gestionar la ventana

ESTRATEGIA	CUÁNDO USARLA
Curar, no acumular	Siempre. Solo los documentos y el historial que de verdad hacen falta ahora.
Documento arriba, pregunta abajo	Al trabajar sobre un texto largo. Reduce el "lost in the middle".
Pedir citas primero	En análisis de documentos: que extraiga las citas relevantes y luego responda. Lo ancla al texto.
Resumir y reiniciar	Conversación muy larga: pide un resumen del estado y abre un chat nuevo con él.
Una tarea por conversación	Mezclar temas ensucia el contexto. Un hilo por objetivo.

Trata la ventana como un **recurso escaso que tú administras**, no como un cajón infinito. La calidad depende tanto de lo que *quitas* como de lo que pones.

2

TEMA 2

Prompt engineering: la base

No son palabras secretas: es comunicación clara con un colaborador literal.

Claridad y dirección: la regla de oro

MARCO MENTAL

Trata al modelo como **un empleado nuevo, brillante pero sin contexto** sobre tu empresa ni lo que tienes en la cabeza. Muy capaz, pero no adivina. Cuanto más preciso seas, mejor el resultado.

LA REGLA DE ORO (ANTHROPIC)

Muéstrale tu prompt a un colega con poco contexto y pídele que lo siga. **Si se confunde, el modelo también.**

Menos efectivo:

"Crea un dashboard de analítica."

Más efectivo:

"Crea un dashboard de analítica. Incluye tantas funciones e interacciones relevantes como sea posible. Ve más allá de lo básico para lograr una implementación completa."

MATIZ DE 2026

Los modelos actuales siguen instrucciones de forma muy literal. Gritar "**CRÍTICO: DEBES hacer X**" hoy provoca *overtriggering*. Basta lenguaje normal: "*Usa esta herramienta cuando...*".

Dar contexto y motivación

Explicarle **por qué** quieres algo mejora la respuesta: puede generalizar a partir de la intención en vez de seguir la regla al pie de la letra.

Menos efectivo:

"NUNCA uses puntos suspensivos."

Más efectivo:

"Tu respuesta la leerá en voz alta un motor de texto-a-voz, así que nunca uses puntos suspensivos porque el motor no sabe pronunciarlos."

Con el "por qué", el modelo entiende el **objetivo real** y decide bien en casos que tu regla no había previsto. Es la diferencia entre dar una orden y dar contexto a alguien inteligente.

Rol y control del formato

Dar un rol (system prompt)

Una sola frase enfoca tono y comportamiento. En chats va en "instrucciones personalizadas" o en un Proyecto; vía API es el *system prompt*.

```
"Eres un asistente experto
en Python que prioriza
código idiomático y legible.
Explicas el porqué en
una frase."
```

El system prompt define **quién es** el modelo de forma persistente; el mensaje de usuario es la tarea de este turno.

Controlar el formato

- **Di qué hacer, no qué no hacer.**
"Escribe en párrafos de prosa" > "No uses markdown".
- **Usa indicadores con etiquetas.**
"...dentro de <prosa>...</prosa>".
- **Que tu prompt se parezca a la salida.**
Prompt sin markdown → menos markdown.

2026: los modelos son más concisos por defecto. Si quieres un resumen o más detalle, **pídelo explícitamente**.

Anatomía de un buen prompt

No todos los prompts necesitan todas las piezas, pero este es el esqueleto, en un orden que funciona:

- | | |
|-----------------------|--|
| 1 · ROL / CONTEXTO | quién eres y para qué sirve la tarea |
| 2 · INSTRUCCIÓN | qué quieres, en pasos si hace falta |
| 3 · CONTEXTO / DATOS | documentos (¡arriba si son largos!) |
| 4 · EJEMPLOS | 1-5 ejemplos del formato deseado |
| 5 · FORMATO DE SALIDA | cómo quieres exactamente la respuesta |
| 6 · LA PREGUNTA | al final, sobre todo con documentos largos |

Un buen prompt no es largo ni rebuscado: es **claro, específico, estructurado y honesto**. Casi todos los problemas de calidad se resuelven mejorando el prompt antes que cambiando de modelo.

3

TEMA 3

Estrategias de estructuración

Few-shot, chain-of-thought y XML: tres palancas que se potencian.

Few-shot: enseñar con ejemplos

ANALOGÍA · MOSTRAR EN VEZ DE EXPLICAR

La diferencia entre describir cómo doblar una camiseta y doblar dos delante de alguien. Con dos demostraciones capta el patrón al instante. El modelo aprende igual: ve el patrón y lo replica.

Clasifica el sentimiento como POSITIVO, NEUTRO o NEGATIVO.

<ejemplos>

<ejemplo>"Llegó tarde pero el producto está bien." → NEUTRO</ejemplo>

<ejemplo>"Una maravilla, lo recomiendo a todos." → POSITIVO</ejemplo>

<ejemplo>"Se rompió a la semana, pésima calidad." → NEGATIVO</ejemplo>

</ejemplos>

Reseña: "Cumple lo que promete, sin más." →

Cómo deben ser

- **Relevantes** a tu caso real.
- **Diversos**: cubren casos límite.
- **Estructurados** en etiquetas.

Dónde brilla

3–5 ejemplos es el punto óptimo. Ideal para **clasificación, extracción y formato estricto** (JSON, YAML, plantillas).

Chain-of-thought: razonar en pasos

Pedirle que muestre su razonamiento *antes* de la respuesta final. Como construye texto token a token (Módulo 1), darle espacio para "pensar en voz alta" mejora el resultado en tareas de razonamiento.

ANALOGÍA · MOSTRAR EL DESARROLLO

Como exigir a un estudiante el desarrollo y no solo el resultado: comete menos errores, y tú ves dónde se equivocó si falla.

Simple (zero-shot CoT):

"...resuelve el problema. Piensa paso a paso antes de la respuesta final."

Estructurado:

"Razona dentro de <razonamiento> y da la respuesta en <respuesta>."

MATICES

Para razonamiento, el **zero-shot CoT** suele ganarle al few-shot. Los modelos actuales tienen un **modo "thinking" adaptativo** que lo hace internamente. Y pídele que **se autoverifique**: *"antes de terminar, comprueba tu respuesta contra [criterio]"*.

XML tagging: estructurar el prompt

ANALOGÍA · LAS CARPETAS ROTULADAS

Hojas sueltas mezcladas vs. carpetas rotuladas: "instrucciones", "datos", "ejemplos". Con los rótulos nadie confunde un dato con una orden. Las etiquetas XML son esos rótulos.

Separan sin ambigüedad las partes de un prompt complejo. Los modelos Claude fueron entrenados para prestarles atención especial.

Buenas prácticas

- Nombres **consistentes y descriptivos**.
- **Anida** cuando hay jerarquía.
- No hay etiquetas "oficiales": claras y coherentes.

```
<instrucciones>
  Resume el informe en 3 viñetas
  para un comité ejecutivo.
</instrucciones>

<documento>
  {{texto largo del informe}}
</documento>

<formato>
  Tres viñetas, máx. 20 palabras
  cada una, sin jerga técnica.
</formato>
```

El patrón completo

Las tres se potencian: **XML organiza**, **few-shot muestra** el patrón, **CoT da espacio** para razonar.

```
<instrucciones>
  Eres un analista de soporte. Clasifica cada ticket por urgencia
  (ALTA/MEDIA/BAJA) y justifica brevemente.
</instrucciones>

<ejemplos>
  <ejemplo>
    Ticket: "No puedo entrar y tengo una demo con un cliente en 10 minutos."
    <razonamiento>Bloqueo total + límite de tiempo inmediato.</razonamiento>
    <respuesta>ALTA</respuesta>
  </ejemplo>
</ejemplos>

<ticket>"Sería bueno poder exportar a Excel algún día."</ticket>

Razona dentro de <razonamiento> y da el veredicto en <respuesta>.
```

Empieza **simple**; añade estructura solo cuando la respuesta no salga bien a la primera.

4

TEMA 4

Una tarea vs. varias

¿Todo en un prompt, o partirlo en pasos? La decisión del día a día.

Cuándo un prompt hace demasiado

A más restricciones simultáneas, más probable que cumpla unas y "olvide" otras (Módulo 1). Señales de sobrecarga:

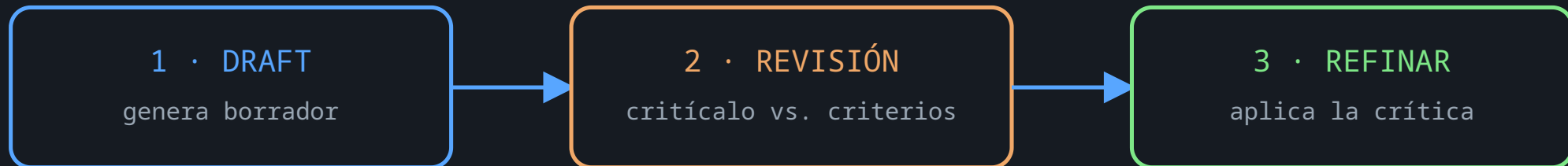
- Pide **varias cosas de naturaleza distinta** a la vez (investigar y redactar y formatear y traducir).
- El resultado cumple unas instrucciones e ignora otras de forma inconsistente.
- Necesitas **inspeccionar un paso intermedio** antes de seguir.
- El prompt es tan largo que ya no sabes qué parte falla.

ANALOGÍA · EL PEDIDO EN EL RESTAURANTE

Puedes pedir "tráeme la carta, recomiéndame, sírvenme el vino y trae la cuenta" de golpe — pero algo se perderá. Paso a paso sale mejor. Con tareas complejas, el modelo agradece lo mismo.

Encadenar pasos: borrador → revisión → refinar

Descomponer en una secuencia donde la salida de un paso alimenta al siguiente. Cada paso es más simple, fiable y fácil de depurar. El patrón estrella es la **autocorrección**:



VENTAJA OCULTA

Puedes **ver y corregir cada paso**. Si el resumen salió mal, lo arreglas ahí sin rehacer toda la cadena. Es depuración aplicada a prompts — el corazón del Módulo 4.

¿Un prompt o varios?

SITUACIÓN	MEJOR ENFOQUE
Resumir un documento	Un solo prompt
Extraer datos a JSON	Un solo prompt (con few-shot)
Investigar, redactar y maquetar un informe	Encadenar pasos
Necesitas validar un paso intermedio	Encadenar pasos
Procesar 50 elementos independientes	Paralelo (uno a uno o por lotes)
Razonamiento complejo de varios pasos	Un prompt con CoT / thinking

REGLA PRÁCTICA

Empieza con **un solo prompt bien escrito**. Divide solo cuando el modelo se atragante, cuando necesites inspeccionar un paso, o cuando una parte falle de forma repetida. No optimices la complejidad antes de necesitarla.

5

TEMA 5

Evaluar si una respuesta es buena

La habilidad que separa al novato del experto: juzgar, no solo generar.

"Suenan bien" no es "es correcto"

Los modelos producen texto fluido y seguro **incluso cuando se equivocan**. Esa fluidez es la trampa: asociamos confianza y buena redacción con veracidad, y el modelo es excelente en ambas por diseño (alucinaciones + RLHF, Módulo 1).

ANALOGÍA · EL CANDIDATO ELOCUENTE

En una entrevista, quien habla con seguridad y nunca duda *parece* el mejor. Pero la elocuencia no es competencia. Verifica lo que dice, no cómo lo dice.

CUIDADO CON LA ADULACIÓN (SYCOPHANCY)

El modelo tiende a darte la razón. Si preguntas "*¿verdad que esto está bien?*", es más probable que diga que sí. Pregunta en neutro: **"Encuentra los problemas de esto"** en vez de "**¿está bien esto?**".

Los cinco criterios

En vez de un juicio global ("me gusta / no me gusta"), evalúa por criterios concretos:

CRITERIO	PREGUNTA DE CONTROL
Correctitud	¿Los hechos, datos y afirmaciones son verdaderos y verificables?
Compleitud	¿Cubre todo lo que pedí, o dejó partes fuera?
Coherencia	¿El razonamiento se sostiene? ¿Hay saltos o contradicciones?
Fidelidad a la fuente	Si di un documento, ¿se basa en él o se lo inventó?
Formato y utilidad	¿Cumple el formato? ¿Me sirve tal cual o tengo que rehacerlo?

Un puntaje global ("7/10") esconde *qué* falló. Evaluar criterio a criterio te dice **exactamente dónde está el problema** — y qué hacer al respecto.

Rúbricas, el modelo como juez, y tu checklist

Rúbricas

Holística: un puntaje global. Rápida pero opaca.

Analítica: un puntaje por criterio. Más trabajo, pero muestra qué mejoró o empeoró. **La recomendada.**

El modelo como juez

Usar un modelo para evaluar la salida de otro contra una rúbrica. Útil a escala, con cautela: dale **criterios específicos** y pídele puntaje **por criterio**. Tiene sesgos (premia lo largo y bien formateado) → primer filtro, no veredicto final.

Checklist de 30 segundos

- ¿Hay datos específicos? → verifícalos por fuera.
- ¿Le di una fuente? → ¿se sostiene en ella?
- ¿Pedí varias cosas? → ¿cumplió todas?
- ¿El razonamiento se sostiene paso a paso?
- ¿Pregunto en neutro o di pistas?
- ¿La acepto porque es correcta, o porque suena bien y estoy cansado?

PRINCIPIO CLAVE

El modelo es un colaborador capaz, no un oráculo. Tu valor no está en generar — sino en **dirigir y verificar**. La evaluación es donde pones ese valor.



SEGUNDA HORA

Práctica y retroalimentación

Con el modelo abierto. El objetivo no es "acertar": es *observar la diferencia*.

Cinco ejercicios guiados

#	EJERCICIO	QUÉ PRACTICA	TIEMPO
1	Antes y después: reescribe un prompt real con rol, instrucción específica, el "por qué" y formato	Claridad (2.1–2.4)	10 min
2	Una clasificación sin ejemplos vs. con 3 ejemplos estructurados	Few-shot (3.1)	10 min
3	Un problema de razonamiento: solo respuesta vs. con <razonamiento>/<respuesta>	Chain-of-thought (3.2)	10 min
4	Redactar algo: borrador → "critica y lista 3 mejoras" → reescribir	Chaining (4.2)	15 min
5	Pedir un dato de nicho, aplicar el checklist, verificar y repreguntar en neutro	Evaluación (5.4)	15 min

Tras cada ejercicio, comparte el resultado y recibe retroalimentación. **Cierre:** ¿qué técnica vas a incorporar a tu trabajo desde mañana?

Los 5 temas, de un vistazo

#	TEMA	IDEA CENTRAL	PARA LLEVAR
1	Ventana de contexto	Espacio finito que todo comparte; el modelo no recuerda entre llamadas	Curar, no acumular. Lo importante al inicio o al final
2	Prompt engineering	Comunicación clara con un empleado brillante sin contexto	Si un colega se confunde, el modelo también. Da el "por qué"
3	Estructuración	Few-shot muestra, CoT razona, XML organiza	Empieza simple; añade estructura solo si hace falta
4	Dividir o no	A más restricciones juntas, más se "olvida"	Un prompt por defecto; encadena para inspeccionar pasos
5	Evaluar	"Suená bien" ≠ "es correcto"; ojo a la adulación	Evalúa por criterios, pregunta en neutro, verifica